Vapor Stone
RAIL SYSTEMS
A Wabtec company

10 655 Henri-Bourassa Blvd., West
Saint-Laurent (Québec) Canada
H4S 1A1

Tel: 514.335.4200
Fax: 514.335.4231

www.wabtec.com

Regional Transportation District

RTD Denver EMU

Project # 7230

Door System

Software Design Description (SDD) Monitoring (MONIT) Module

Contract Number	-	CDRL Number	-
		Customer Ref. Number	TBD

NOTICE
Plans, specifications and engineering data submitted herewith or which may be submitted, remain the property of **Vapor Stone Rail Systems** and shall not be reproduced or altered or used in whole or in part except by express consent of **VSRS**.

AVIS
Les plans, spécifications et données techniques soumis ci-joints, ou qui peuvent être soumis, restent la propriété de **Vapor Stone Rail Systems** et ne doivent pas être reproduits, altérés ou utilisés, totalement ou partiellement, sans le consentement express de **VSRS**.

Prepared : James Kim — Software Designer — James Kim 2011-10-06

Verified : Martin Ouellet, Eng. — Software Department Supervisor — 2011-10-06

Approved : Aldo Di Rocco — Project Manager — For Aldo Di Rocco 2011-10-07

Name / Title **Signature / Date**

Document No.	**201111630**	Revision	**1**
		Date (yyyy-mm-dd)	**2011-10-05**

Revision List

Rev.	Date	By	ECD	Description of changes
1	2011-10-05	James Kim	-	Official release.
0.1	2011-10-05	James Kim	-	Implementing comments from internal inspection I-1852.
0	2011-09-19	James Kim	-	Original issue.

Preface

This Software Design Description (SDD) complies with the requirements of IEEE Std. 1016-1998 and IEEE Std. 1558-2004.

Table of Contents

Revision List 2
Preface....2
Table of Contents 3
List of Figures 4
List of Tables 4

1 INTRODUCTION 5

1.1 Purpose 5
1.2 Scope....5
1.3 Definitions and Acronyms....5
1.3.1 Definitions 5
1.3.2 Acronyms 5
1.4 Document Organization....6

2 REFERENCES 7

3 SOFTWARE DESIGN....8

3.1 Decomposition Description....8
3.1.1 Module Decomposition 8
3.1.2 Concurrent Process Decomposition 8
3.1.3 Data Decomposition....8
3.2 Dependency Description 10
3.2.1 Inter-Module Dependencies....10
3.2.2 Inter-Process Dependencies 11
3.2.3 Data Dependencies 11
3.3 Interface Description....12
3.3.1 Module Interfaces 12
3.4 Detailed Design 15
3.4.1 MONIT Module Detailed Design 15
3.4.2 Data Entities Detailed Design 25

List of Figures

Figure 3-1: Inter-Module Dependencies Diagram 10
Figure 3-2: Inter-Process Dependencies Diagram 11
Figure 3-3: MONIT_DisableCommand Flowchart 16
Figure 3-4: monit_Task Flowchart 17
Figure 3-5: monit_FaultsDetection Flowchart 18
Figure 3-6: monit_FaultLogger Flowchart 20
Figure 3-7: monit_DoorOperatorCommand Flowchart 21
Figure 3-8: State Transition Diagram of Door Open Without Command 24

List of Tables

Table 3-1: Structure of Event log counter 8
Table 3-2: Event Description File Structure 9

1 INTRODUCTION

This document is the Software Design Description of the Monitoring (MONIT) Module developed by Vapor Stone Rail Systems.

The MONIT Module is a part of the Door Control Unit (DCU) Application that manages the received door command and monitoring.

1.1 Purpose

The purpose of this Software Design Description (SDD) is to describe the software components and required implementation of the MONIT Module.
The intended audience is the Vapor Stone Rail Systems software development team and maintenance team members of the transit authority and the car builder.

1.2 Scope

The MONIT Module is a software module used for processing the received door command and monitoring the inputs; therefore, it is safety related module. This module interfaces with other modules.

1.3 Definitions and Acronyms

1.3.1 Definitions

Door Control Unit: A specific processor based electronic module running a Door Control Unit Application. It is used for controlling and monitoring the door.

1.3.2 Acronyms

Acronym	Meaning
ASCII	American Standard Code for Information Interchange
CDRL	Contract Data Requirements List
CL	Car Line
CPU	Central Processing Unit
CRC	Cyclic Redundancy Check
DCMLN	Door Cut-out Mechanical Lock in Normal
DCU	Door Control Unit
DOP	Door Operator
DSR	Door Summary Relay
ECD	Engineering Change Document
EDF	Event Description File
FLOG	Fault Logger
I/O	Input and Output
IEEE	Institute of Electrical and Electronics Engineers
LDTS	Local Diagnostic and Test System
LH	Left Hand
LSS	Lock Sensing Switch
MHz	MegaHertz
MONIT	Monitoring
ms	Millisecond
NVRAM	Non-Volatile Random Access Memory
OC	Over Current
OS	Operating System
PID	Proportional Integral Derivative
PSS	Panel Sensing Switch
PWM	Pulse Width Modulation
RAM	Random Access Memory
RH	Right Hand
RTN	Return

Acronym	Meaning
SCI	Software Configuration Items
SDD	Software Design Description
SRAM	Static Random Access Memory
SRS	Software Requirements Specification
Std	Standard
TL	Train Line
VSRS	Vapor Stone Rail Systems
W/O	Without

1.4 Document Organization

This SDD is divided into three sections and the following specific information is provided within each one:

- Section 1, Introduction, states the purpose, identifies the scope, gives definitions, abbreviations and an overview of the SDD.
- Section 2, References, lists the factors that affect the product and its requirements.
- Section 3, Software Design, summarizes followings:
    - Decomposition Description: describes functions of software entities.
    - Dependency Description: gives a brief description of the dependency between each entity.
    - Interface Description: describes the interface of the entity.
    - Detailed Design: details the functions implemented by each software entity, presents and explains tasks performed by each software entity in functional conditions.

Within the document, diagrams, flowcharts and tables are used to present the software design:

- Decision flowcharts give a graphical view of how entities and functions process work.
- The diagram Figure 3-1 describes the dependency between different modules.
- Tables contain data structures used by the MONIT Module. They also present C code.

2 REFERENCES

This section lists the number, title, revision, date, and source of all documents referenced in this SDD.

Standards

IEEE Std 1016-1998	IEEE Recommended Practice for Software Design Descriptions, September 23rd 1998, IEEE, www.ieee.org
IEEE Std 1558-2004	IEEE Recommended Practice for Software Documentation for Rail Equipment and Systems, March 23rd 2005, IEEE, www.ieee.org

Vapor Stone Rail Systems Documents

201111208	Software Quality Assurance Plan, Door Control System, Rev. 0
201111270	Software Requirements Specification, Door Control System - DCM Application, Rev. 1
201111576	Software Application Configuration Specification, Rev. 1
S17-07	Software Design Description Template, Rev. 0, 2002-01-15[1]

[1] Complemented with IEEE STD 1558 Requirements

3 SOFTWARE DESIGN

3.1 Decomposition Description

3.1.1 Module Decomposition

3.1.1.1 MONIT Module Description

The MONIT Module is a single entity module performing following functions:

- Monitor and verify the DCU operation and faulty conditions
- Manage the door operation commands
- Activate the outputs as commanded

3.1.2 Concurrent Process Decomposition

3.1.2.1 IO Task Description

The MONIT task loop runs every 50ms and calls the internal functions. This task is running as an independent thread.

3.1.3 Data Decomposition

3.1.3.1 IO Data Description

The MONIT Module uses the following structure to manage the maximum number of events set per day.

Table 3-1: Structure of Event log counter

Identification	EventLogCounter
Entity Type	struct CounterStruct

Prototype	`// Maximum number of fault in the system`
	`#define FAULT_MAX_NUMBER  256`
	
	`// Structure containing the number of event in a day`
	`typedef struct _CounterStruct`
	`{`
	`  unsigned long  ulTimeStamp;  // Time Stamp of the Event, 4 bytes`
	`  unsigned char  ucEventCounter[FAULT_MAX_NUMBER]; // Counter`
	`  // associated with the fault number`
	`  unsigned int  uiCrc;  // CRC of the 2 previous members`
	`  // ulTimeStamp and ucEventCounter[]`
	`}CounterStruct;`
	
	`CounterStruct huge EventLogCounter;`

The MONIT Module uses the following structure to consult the attributes and type of the event.

Table 3-2: Event Description File Structure

Identification	EDF
Entity Type	struct EdfRecordStruct
Prototype	

```
#define NUMBER_OF_EVENT             128
#define MAX_EVENT_DESC_SIZE         45
#define MAX_DIAGNOSTIC_STRING_LEN   745

//*********************************************************************
// Event Description File ( EDF ) format definition.
//*********************************************************************
// Header definition within the EDF file.
typedef struct Bbd_EdfFileHeaderStruct
{
    unsigned char   ucMajorEdfVersion;  // Major version of the EDF.
    unsigned char   ucMinorEdfVersion;  // Minor version of the EDF.
    unsigned char   ucDescSizeDivBy5;   // One 5th of the Description size.
    unsigned char   ucDiagSizeDivBy5;   // One 5th of the Diagnostic size.
} Bbd_EdfFileHeaderStruct;

// The Alignement of the following structures are per byte.
#pragma pack(1)
//#pragma BYTEALIGN

// Record definition within the EDF file.
typedef struct Bbd_EdfRecordStruct
{
    unsigned int    uiEventCode;    // Event code.
    unsigned int    uiAttributes;   // Event code attributes
    unsigned char   ucType;         // Type of the event Bit 0
                                    // 0= Pulse Type,
                                    // 1= Set / Reset Type
    unsigned char   ucDescription[MAX_EVENT_DESC_SIZE]; // Null terminated
                                                        // description of
                                                        //the associated event
    unsigned char   ucDiagnostic[MAX_DIAGNOSTIC_STRING_LEN];
} Bbd_EdfRecordStruct;

// Record definition within the EDF file.
typedef struct EdfRecordStruct
{
    Bbd_EdfFileHeaderStruct Edf_Version;
    Bbd_EdfRecordStruct Edf_Struct[NUMBER_OF_EVENT];
} EdfRecordStruct;

const EdfRecordStruct xhuge EDF;
```

3.2 Dependency Description

3.2.1 Inter-Module Dependencies

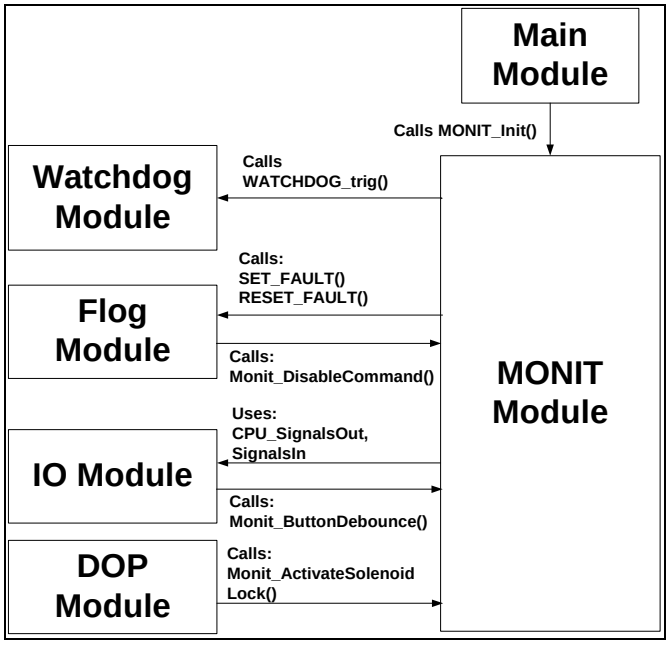

Figure 3-1: Inter-Module Dependencies Diagram

3.2.1.1 MONIT Module

Figure 3-1 shows the Inter-Module Dependencies Diagram of the MONIT Module. It displays MONIT Module's dependency over other modules.

3.2.2 Inter-Process Dependencies

As seen in Figure 3-2, the monit_FaultLogger function of the MONIT Module waits for the message from the FLOG Module for the event code number.
The monit_DoorOperatorCommand function sends the door command message to the DOP Module.
The monit_Task waits for the OS Semaphore “MoniTaskSem” before going to loop.

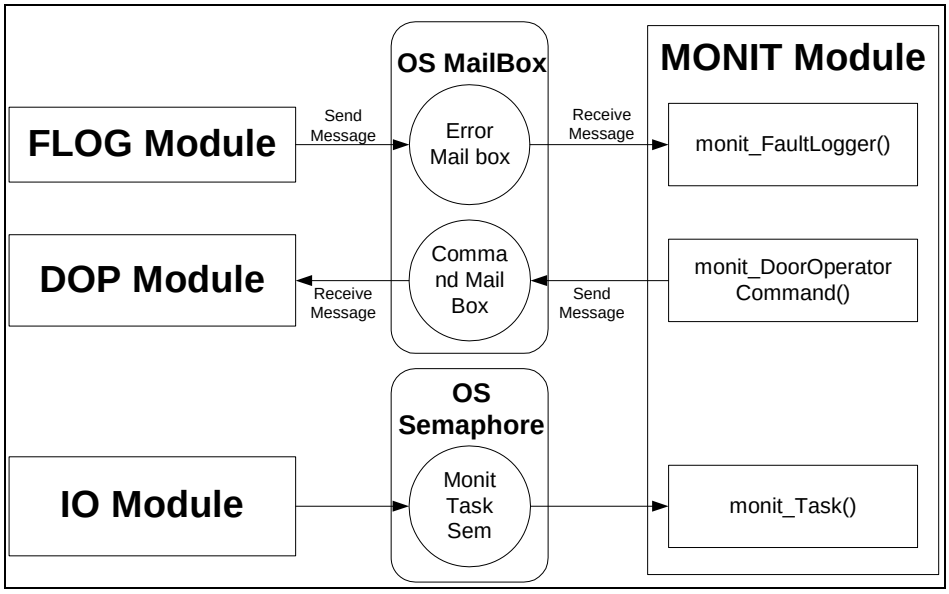

Figure 3-2: Inter-Process Dependencies Diagram

3.2.3 Data Dependencies

As seen in Figure 3-1, MONIT Module uses the following data from the IO Module:

- SignalsIn: This is a structure that contains the status of all inputs.
- CPU_SignalsOut: This is a structure that contains the status of all outputs.

3.3 Interface Description

This section describes the functions of MONIT Module, listed in alphabetical order.

3.3.1 Module Interfaces

Before starting to use this module, the Main Module must call MONIT_Init function first.

3.3.1.1 MONIT Module External Interfaces

Function Name	void MONIT_ActivateSolenoidLock(unsigned int uiPWM)
Description	This function energizes the solenoid lock.
Input	uiPWM: This variable is the percentile of the pulse width modulation.
Output	None.
Return	None.

Function Name	void MONIT_ButtonDebounce(void)
Description	This function filters out the unstable signals before the monitoring.
Input	None.
Output	None.
Return	None.

Function Name	void MONIT_DisableCommand(unsigned int uiMsg)
Description	This function manages the counter for the disable command when set event has an attribute of disable command. It consults the Table 3-2 for the attribute of the event.
Input	uiMsg: This variable is the number of the event code.
Output	None.
Return	None.

Function Name	void MONIT_Init(void)
Description	This function initializes the variables and creates monit_Task.
Input	None.
Output	None.
Return	None.

3.3.1.2 MONIT Module Internal Interface

Function Name	void monit_DoorCommand(unsigned char *pucCommand)
Description	This function monitors train lines and manages the door command. It updates the pucCommand with either open or close.
Input	None.
Output	pucCommand: This is a door command.
Return	None.

Function Name	void monit_DoorOperatorCommand(void)
Description	This function manages all door commands and send the message to the DOP Module using the OS MailBox as seen in the Figure 3-2.
Input	None.
Output	None.
Return	None.

Function Name	void monit_EmergencyCommand(unsigned char *pucCommand)
Description	This function monitors the emergency handle and manages door command. It updates the pucCommand with either neutral, open or close.
Input	None.
Output	pucCommand: This is a door command.
Return	None.

Function Name	void monit_EncoderCommand(unsigned char *pucCommand)
Description	This function monitors the encoder status, and if there is a problem with encoder, it updates the pucCommand with error command.
Input	None.
Output	pucCommand: This is a door command.
Return	None.

Function Name	void monit_EncoderDetect(void)
Description	This function monitors the encoder if there is an encoder lost and the mismatch of encoder.
Input	None.
Output	None.
Return	None.

Function Name	void monit_FaultLogger(void)
Description	This function manages the maximum number of the event logging per day, and it starts recording the snapshots if the set event requires it. It consults the Table 3-2 for the type and attribute of the event. It also wait for the message from the OS MailBox as seen in the Figure 3-2.
Input	None.
Output	None.
Return	None.

Function Name	void monit_FaultsDetection(void)
Description	This function calls other internal monitoring functions.
Input	None.
Output	None.
Return	None.

Function Name	void monit_InterlockPowerDetect(void)
Description	This function monitors the motor open interlock and the solenoid interlock supplies.
Input	None.
Output	None.
Return	None.

Function Name	void monit_MechanicalLockCommand(unsigned char *pucCommand)
Description	This function monitors the mechanical lock input and manages door command. It updates the pucCommand with either neutral or the emergency close.
Input	None.
Output	pucCommand: This is a door command.
Return	None.

Function Name	void monit_MotorOverCurrentDetect(void)
Description	This function monitors the over current shutdown activation.
Input	None.
Output	None.
Return	None.

Function Name	void monit_Output(void)
Description	This function manages output of the controller. It uses CPU_SignalsOut from the IO Module.
Input	None.
Output	None.
Return	None.

Function Name	void monit_OpenWithoutCommand(unsigned char *pucCommand)
Description	This function monitors if doors are open without the proper command. If this is the case, it will update the pucCommand with emergency close.
Input	None.
Output	pucCommand: This is a door command.
Return	None.

Function Name	void monit_SupplyVoltage(void)
Description	This function monitors the power supply.
Input	None.
Output	None.
Return	None.

Function Name	void monit_Task(void)
Description	This function runs on a for loop every 50ms. It calls other internal functions.
Input	None.
Output	None.
Return	None.

3.4 Detailed Design

This section details all operations performed by MONIT Module.

3.4.1 MONIT Module Detailed Design

The following sub-sections describe each function.

3.4.1.1 MONIT_Init Function

This function initializes all variables and resets door command. It then calls the monit_Task function.

3.4.1.2 MONIT_ButtonDebounce Function

This function detects transitions in the inputs used by monitoring and sets a transition flag to avoid making a diagnostic on an unstable signal.
The following signal is considered unstable for one cycle (50 ms) upon detection of a transition:

- Emergency Handle

The following signals are considered unstable for 200 ms upon detection of a transition:

- Mechanical Lock
- Panel Sensing Switch (PSS) Left, PSS Right, Lock Sensing Switch (LSS) Left, and LSS Right

The following signals are considered unstable for 500 ms upon detection of a transition:

- 5V Logic Supply
- 5V Encoder Supply
- 15V Motor Supply
- Motor Power Sense
- V-Gate Open Power
- V-Gate Close Power

3.4.1.3 MONIT_ActivateSolenoidLock Function

This function energizes the solenoid lock with the input parameter uiPWM.

3.4.1.4 MONIT_DisableCommand Function

This function disables or enables the door commands depending on the attributes of each event. (See SACS document for the attribute “Disable Open” of event) Figure 3-3 shows the process done:

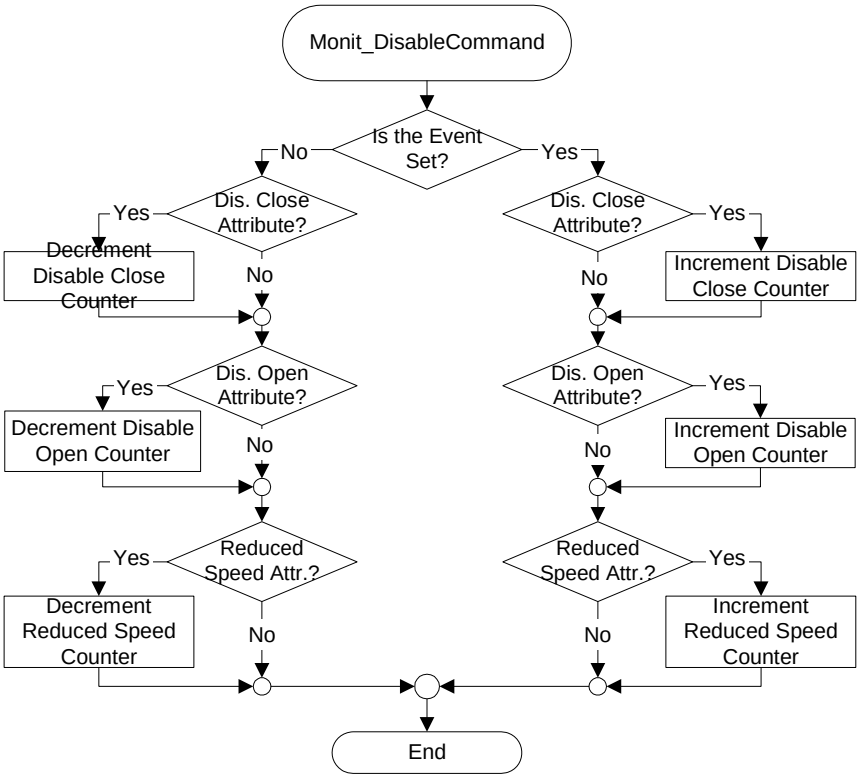

Figure 3-3: MONIT_DisableCommand Flowchart

3.4.1.5 monit_Task Function

This function manages the fault logger, activates outputs, updates the Operating Status and controls the Door Operator module. This task is scheduled each 50ms and calls other functions of the Monitoring module. Figure 3-4 shows the process.

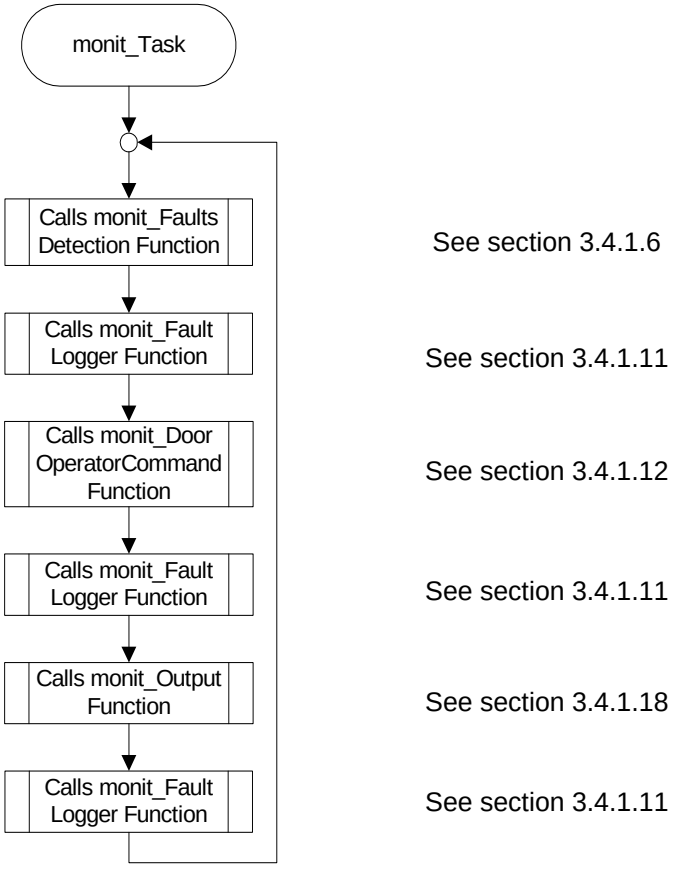

Figure 3-4: monit_Task Flowchart

3.4.1.6 monit_FaultsDetection Function

Figure 3-5 describes this function:

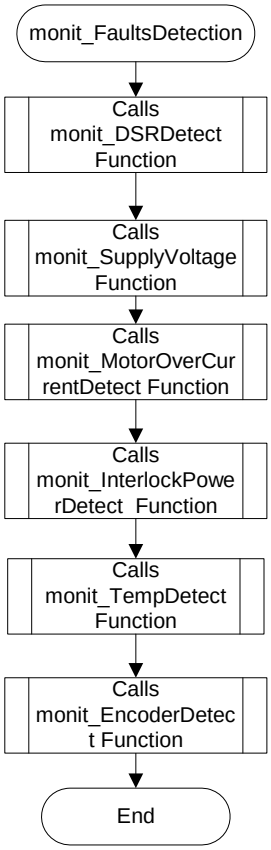

Figure 3-5: monit_FaultsDetection Flowchart

3.4.1.7 monit_DSRDetect Function

This function verifies the Door Summary Relay (DSR) high and low side monitoring with the PSS left and right and LSS left and right. If two PSS and two LSS indicate open and unlocked with DSR low and high energized then it sets the event 71 “Door Summary Relay High Error” and 72 “Door Summary Relay Low Error”. The DCU resets the event 71 or 72 if DSR monitoring signal becomes consistent with the PSS and LSS.

3.4.1.8 monit_SupplyVoltage Function

This function verifies following:

Power Supply Monitoring:
Whenever the Motor Power Sense is deactivated, the DCU sets the Event 7, Motor Power not available. The Event 7 is reset when the Motor Power Sense is activated.
While the Motor Power is available, if the Motor Power input is deactivated for 3 seconds, the DCU sets Event 8, Motor Power Supply Below 30V. The Event 8 is reset when the Motor Power is activated for 3 seconds or the Motor Power Sense is deactivated.

5V Logic Supply Monitoring:
Whenever the Voltage Monitoring 5V Logic Supply is out of range, the DCU sets the Event 39, 5V Logic Supply Out of Range.
The Event 39 is reset when the Voltage Monitoring 5 V Logic Supply is within tolerances.

5V Encoder Supply Monitoring:
Whenever the Voltage Monitoring 5V Encoder Supply is out of range, the DCU sets the Event 40, 5V Encoder Supply Out of Range.
The Event 40 is reset when the Voltage Monitoring 5V Encoder Supply is within tolerances.

15V Motor Supply Monitoring:
Whenever the Voltage Monitoring 15 V Motor Supply is out of range, the DCU sets the Event 41, 15V Motor Supply Out of Range.
The Event 41 is reset when the Voltage Monitoring 15V Motor Supply is within tolerances.

V-Gate Open Supply Monitoring:
Whenever the Voltage Monitoring V-Gate Open Supply is out of range, the DCU sets the Event 42, V-Gate Open Supply Out of Range.
The Event 42 is reset when the Voltage Monitoring V-Gate Open Supply is within tolerances.

V-Gate Close Supply Monitoring:
Whenever the Voltage Monitoring V-Gate Close Supply is out of range, the DCU sets the Event 43, V-Gate Close Supply Out of Range.
The Event 43 is reset when the Voltage Monitoring V-Gate Close Supply is within tolerances.

Analog Reference 2.5V Monitoring:
Whenever the analog reference value is more than 5% off, the DCU sets the Event 87, Analog Reference Error.
The Event 87 is reset when the analog reference value is less than 3% off.

3.4.1.9 monit_MotorOverCurrentDetect Function

When the Motor Power is available for more than 1.5 seconds, the current limit test in not running and the Overcurrent Shutdown bit is set, the DCU sets the Event 38 Motor OC Shutdown Activated. The event 38 is reset when the Motor Power is available for more than 1.5 seconds, the current limit test in not running and the Overcurrent Shutdown bit is reset.

3.4.1.10 monit_InterlockPowerDetect Function

This function verifies following:

Motor Open Supply Monitoring:
This function sets the Event 23, Motor Open Supply Error if the Motor Open (H-Bridge) Supply is still activated 2 seconds after two LSS are deactivated from the door closing command.
The Event 23 is reset when Door Open (H-Bridge) Supply becomes inactive with two LSS becoming inactive.

Solenoid Lock Supply Monitoring:
In this function, DCU sets the Event 24, Solenoid Lock Supply Error, if the Feedback Solenoid Interlock Supply stays activated for more than 2 ms after locked and closed doors.
The Event 24 is reset when the Feedback Solenoid Interlock Supply becomes inactive with locked and closed doors.

3.4.1.10.1 monit_EncoderDetect Function

The Motor Encoder Phase Lost fault is detected when a door movement is initiated, the PWM is between 1 and 90% and there is not more than 20 pulses on one phase and none on the other phase, the Event 27, Motor Encoder Phase Lost Error is set. The Event 27 is reset whenever there are pulses on both phases.
The Event 28, Motor Encoder Cable/Circuit Error, is set when the encoder phase A, phase B, and index are activated simultaneously; and is reset otherwise.
The Event 30, Motor Drive Error or Wire Disconnected, is set when the PWM is superior or equal to 60%, the motor current is less than 0.5 A, and there is no encoder pulse for more than 200 ms; and is reset otherwise.

3.4.1.11 monit_FaultLogger Function

Figure 3-6 describes this function:

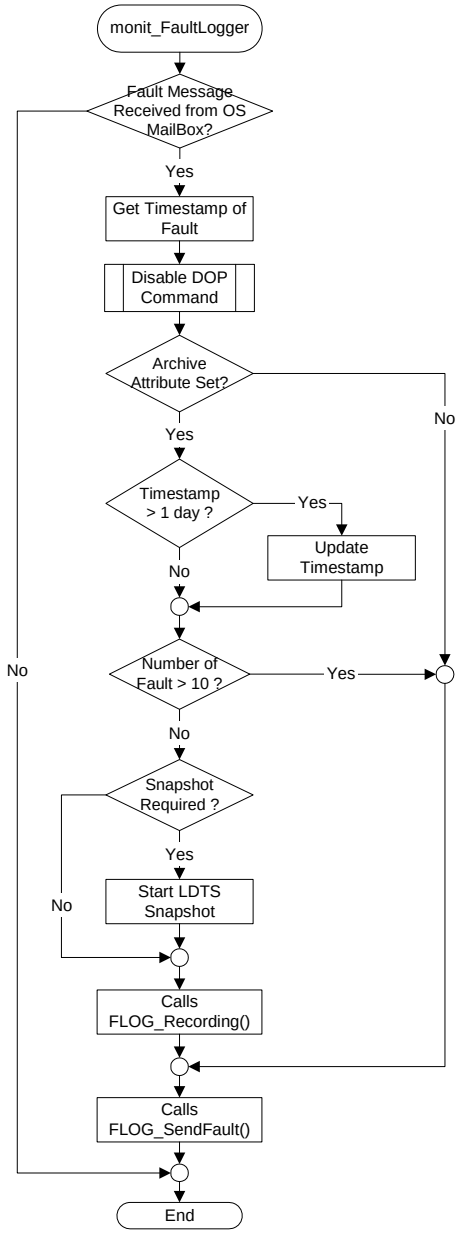

Figure 3-6: monit_FaultLogger Flowchart

Event Log Counter Verification:

This function logs maximum of 10 same events (set) per day. If the maximum has been reached, no event and no snapshot will be recorded. It uses a table to store the counter for each event. The structure of this table is shown in Table 3-1.

- 4 bytes for the timestamp
- 1 byte for each event counter corresponding to a fault
- 2 bytes for the CRC

128 bytes of memory space are reserved for the Event Counters, 4 bytes for timestamp, 1 byte for each event counter corresponding to a fault and 2 bytes for the CRC for a total of 134 bytes. The Timestamp is compared with the actual time to know when to reset the event counters. The event counters are reset everyday by the Monitoring module. When the timestamp is reset, the new time is stored over the old timestamp and the counters are returned to zero.

3.4.1.12 monit_DoorOperatorCommand Function

This function is used to send commands to the door operator module. The commands can be one of the following:

- Door Open Command,
- Door Close Command,
- Emergency Handle Command,
- Mechanical Lock Command,
- Encoder Error Command.

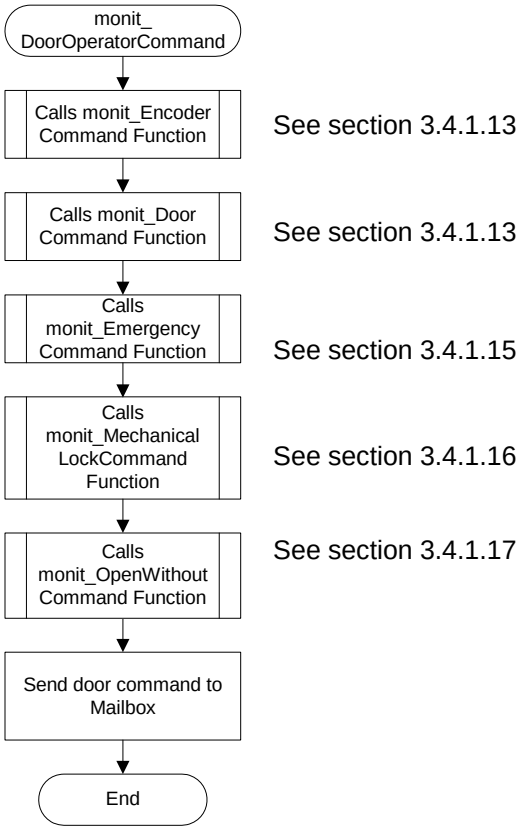

Figure 3-7: monit_DoorOperatorCommand Flowchart

This function sends a message to the Door Operator module through a mailbox.

3.4.1.13 monit_EncoderCommand Function

When one of the encoder faults is detected while commanded to open, a Neutral command is sent to the Door Operator module; otherwise, an Encoder Error (Close with a fixed PWM value) command is sent to the Door Operator. The Neutral command removes power to the motor thus enabling it to move freely.
Those faulty conditions are detected by the PID_Encoder_Test found in PID Module and are logged by the monit_EncoderDetect function found in section 3.4.1.10.1.

3.4.1.14 monit_DoorCommand Function

This function manages door commands. The following events are set if door commands are issued without the presence of the both signals Zero Speed (+) and Zero Speed (-) CL:

- Event 74 Crew Switch w/o Zero Speed
- Event 75 Door Open PWR TL w/o Zero Speed
- Event 76 Door Open RTN TL w/o Zero Speed
- Event 79 Passenger Door Enable TL w/o Zero Speed

They are reset next time Zero Speed (+) and Zero Speed (-) CL signals are present with the door command.

This function also verifies whether the Crew Switch signals are generated in R2 and L2 locations. If this is the case, DCU sets Event 67, Crew Switch Signal at Invalid Location. DCU resets event 67 when Crew Switch signals are inactive.

The following commands are verified in the function:

- Door Open TL Command,
- Crew Switch Door Open Command,
- Passenger Door Enable Command,
- Passenger Door Open Command,
- Door Close TL Command,
- Crew switch Door Close Command,
- Passenger Door Close Command.

Door Open TL Command:
The following condition is required to issue the door open command to the door operator module:

- Both *Zero Speed (+) CL* and *Zero Speed (-) CL* signals stay active;
- *Emergency Handle* signal stays inactive;
- *Door Cut-out Mechanical Lock Switch In Normal* signal stays active;
- Both *Door Open (+) TL* and *Door Open (-) TL* signals are pulsed ;
- The “Disable Open Counter” is zero (See Section 3.4.1.4).

If there is signal mismatch between Door Open (+) and Door Open (-) TL, the DCU sets Event 13, Door Open TLs Mismatch, and the Door Open command is not issued. The Event 13 is reset the next time both Train Lines are activated together.
DCU also verifies if Door Open (+) TL signal is present without the signal Door Cut-out Mechanical Lock In Normal. In this case, DCU sets Event 83 Door Open TL w/o Mechanical Lock Normal. DCU resets it when either Door Open (+) TL becomes inactive or the signal Door Cut-out Mechanical Lock In Normal becomes active.

Crew Switch Door Open Command:
The following condition is required to issue the door open command to the door operator module:

- Both *Zero Speed (+) CL* and *Zero Speed (-) CL* signals stay active;
- *Emergency Handle* signal stays inactive;
- *Door Cut-out Mechanical Lock Switch In Normal* signal stays active;
- Both *Crew Switch Door Open* and *Crew Switch Door Enable* signals are pulsed ;
- The “Disable Open Counter” is zero (See Section 3.4.1.4).

If there is signal mismatch between Crew Switch Door Open and Crew Switch Door Enable, the DCU sets Event 70, Crew Switch Open Signals Mismatch, and the Door Open command is not issued. The Event 70 is reset when both signals are activated together.

Passenger Door Enable Command:
The following condition is required to issue the passenger door enable command to the same function and to the IO module for the Passenger Pushbutton Door Enable Light:

- Both *Zero Speed (+) CL* and *Zero Speed (-) CL* signals stay active;
- *Emergency Handle* signal stays inactive;
- *Door Cut-out Mechanical Lock Switch In Normal* signal stays active;
- Both *Passenger Door Enable (+) TL* and *Passenger Door Enable (-) TL* signals stay active;
- The "Disable Open Counter" is zero (See Section 3.4.1.4).

If there is signal mismatch between Passenger Door Enable (+) and Passenger Door Enable (-) TL, the DCU sets Event 77, Passenger Door Enable TL Mismatch, and the Door Open command is not issued. The Event 77 is reset the next time both Train Lines are activated together.

Passenger Door Open Command:
The following condition is required to issue the passenger door open command to the door operator module:

- Passenger Door Enable command is issued (as above);
- *Passenger Door Open Pushbutton* signal is pulsed.

Door Close TL Command:
The following condition is required to issue the door close command to the door operator module:

- *Emergency Handle* signal stays inactive;
- *Door Cut-out Mechanical Lock Switch In Normal* signal stays active;
- *Door Close TL* signal is pulsed;
- The "Disable Close Counter" is zero (See Section 3.4.1.4).

Crew Switch Door Close Command:
The following condition is required to issue the door close command to the door operator module:

- *Emergency Handle* signal stays inactive;
- *Door Cut-out Mechanical Lock Switch In Normal* signal stays active;
- *Crew Switch Door Close* signal is pulsed;
- The "Disable Close Counter" is zero (See Section 3.4.1.4).

Passenger Door Close Command:
The following condition is required to issue the passenger door close command to the door operator module:

- *Emergency Handle* signal stays inactive;
- *Door Cut-out Mechanical Lock Switch In Normal* signal stays active;
- The configured door open duration for the passenger door enable is over;
- The "Disable Close Counter" is zero (See Section 3.4.1.4).

3.4.1.15 monit_EmergencyCommand Function

When an *Emergency Handle* signal becomes active, DCU sets Event 11, Emergency Handle Activation. Also it sends an Emergency Handle command to the Door Operator module.
Since the emergency handle is spring loaded, it returns to its original position. At same time, DCU resets Event 11.

3.4.1.16 monit_MechanicalLockCommand Function

This function detects activation and reset *Door Cut-out Mechanical Lock Switch In Normal* (DCMLN) signal.

Neutral Command:
A Neutral (Stop and ignore encoder value) command is sent to the Door Operator module when the following condition is met:

- DCMLN signal becomes inactive (either in OFF or CUTOUT position).

DCU sets Event 9, Door Cut-Out Mechanical Lock in Not Normal. When DCMNL signal becomes active (in NORMAL position), DCU resets Event 9.

3.4.1.17 monit_OpenWithoutCommand Function

This function detects any door opening without any command, and it monitors the loss of zero speed with door open.
Figure 3-8 shows the state diagram. It first verifies if a Door Open command is sent to door operator.
The state machine is not executed when there is any transition on lock or panel sensing switches. The state machine is normally in state #1, Door Open state, not closed or not locked. When the current state is #1 and the door returns to fully closed and locked position, it returns to state #2, Door Fully Closed and Locked. If the door unlocked or opened neither with an electrical command nor Emergency Handle, the Event 17, Loss of "Closed&Locked" Status w/o Door Command is set and Emergency Handle command is sent to the Door Operator module.
Upon reception of door open command, the state machine goes from state #2 to state #3, Door Unlocking. The state machine stays in state #3 until the door unlocks or the 1 sec. timeout elapsed. When the door unlocks, the state machine is going to state #1, Door Open. The state machine returns into state #2 whenever the door is not unlocked and timeout elapsed.

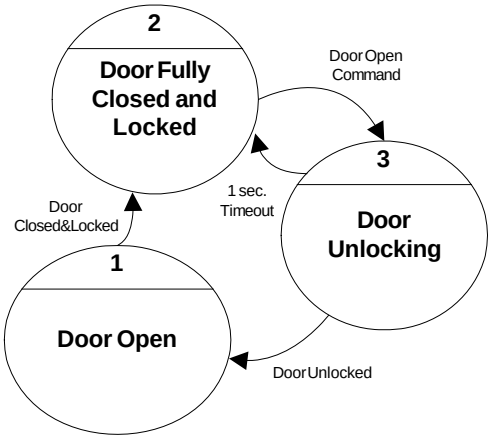

Figure 3-8: State Transition Diagram of Door Open Without Command

In the same time, when DCU detects sudden loss of signals Zero Speed (+) and Zero Speed (-) CL in state #1, it sets the Event 20, Loss of Zero Speed with Door Open.

3.4.1.18 monit_Output Function

This function manages following outputs:

Door Fault LED
The Door Fault LED is activated when at least one fault is in "set" state.

Door Warning LED
The Door Warning LED is activated when at least one exception is in "set" state.

LED Test
The LED Test output is activated only when the Test Switch input is activated and the door is closed and locked.

Door Closing Indicators
Door closing indicators are composed of two components: Door Closing Visual Alarm and Door Closing Audible Alarm. DCU turns on both components when Door Close TL Command is issued from the function monit_DoorCommand (Section 3.4.1.14) until the door is fully closed and locked.

Solenoid Lock Low Side Control
The solenoid lock low side control is activated when the Door operator module initiates the door unlock sequence (not in this function). The solenoid lock output is deactivated after a timeout of 1.4 second elapses or when the PSS indicated that the door is open and a minimum of 0.3 second of activation has expired.

3.4.2 Data Entities Detailed Design

Not applicable.